

Lab-3

Task-1– Basic Encryption and Decryption

Step 1 – Create a sample file

echo "This is a test file.

Line two of the file.

Line three of the file." > plain.txt

Step 2 – Encrypt it using AES and Blowfish

AES-CBC

command (openssl enc -aes-128-cbc -e -in plain.txt -out aes128_cbc.bin \ -K 00112233445566778889aabbccddeeff -iv 0102030405060708) where -e means encrypt, -K is hex key, -iv is initialization vector.
-in = input file, -out = output file.

Observation: Produces binary ciphertext.

AES-CFB

same command

Observation: Stream-like; decrypt exactly recovers text.

BF-CBC

openssl enc -bf-cbc ...

Works similarly to AES but different cipher.

Step 3 – Decrypt one to check

openssl enc -aes-128-cbc -d -in aes128_cbc.bin -out recovered.txt \ -K 00112233445566778889aabbccddeeff -iv 0102030405060708

cat recovered.txt

Task 2 – ECB vs CBC Encryption Mode (3 Marks)

Objective

We will **encrypt an image file (pic_original.bmp)** using **ECB** and **CBC** modes. Then, we will **view the encrypted file as an image** — but we must **keep the first 54 bytes (BMP header) unchanged**, otherwise the image viewer will not open it.

Step 1: Required Files & Tools

pic_original.bmp

Step 2: Encrypt with ECB

```
openssl enc -aes-128-ecb -e \  
-in pic_original.bmp \  
-out pic_ecb_enc.bin \  
-K 00112233445566778889aabbccddeeff \  
-nosalt
```

Step 3: Encrypt with CBC

```
openssl enc -aes-128-cbc -e \  
-in pic_original.bmp \  
-out pic_cbc_enc.bin \  
-K 00112233445566778889aabbccddeeff \  
-iv 0102030405060708 \  
-nosalt
```

Result:

ECB encrypts identical plaintext blocks into identical ciphertext blocks; thus, image patterns leak.

CBC chains blocks with IV and previous ciphertext, hiding patterns completely.

Task3- Encryption mode –corrupted cipher text

This task shows how bit errors in ciphertext affect decrypted plaintext differently in each mode (ECB, CBC, CFB, OFB).

Encryption

KEY=00112233445566778889aabbccddeeff

IV=0102030405060708

```
openssl enc -aes-128-ecb -e -in plain.txt -out plain_ecb.bin -K $KEY -nosalt
```

```
openssl enc -aes-128-cbc -e -in plain.txt -out plain_cbc.bin -K $KEY -iv $IV -nosalt
```

```
openssl enc -aes-128-cfb -e -in plain.txt -out plain_cfb.bin -K $KEY -iv $IV -nosalt
```

```
openssl enc -aes-128-ofb -e -in plain.txt -out plain_ofb.bin -K $KEY -iv $IV -nosalt
```

```
echo "Encryption complete!"
```

```
ls -lh *.bin
```

Copy + error bit

```
cp plain_ecb.bin plain_ecb_corrupt.bin
```

```
cp plain_cbc.bin plain_cbc_corrupt.bin
```

```
cp plain_cfb.bin plain_cfb_corrupt.bin
```

```
cp plain_ofb.bin plain_ofb_corrupt.bin
```

```
for file in plain_*_corrupt.bin; do
```

```
    printf '\x80' | dd of="$file" bs=1 seek=30 conv=notrunc 2>/dev/null
```

```
    echo "Bit flipped: $file"
```

```
done
```

Decrypt

KEY=00112233445566778889aabbccddeeff

IV=0102030405060708

```
openssl enc -aes-128-ecb -d -in plain_ecb_corrupt.bin -out dec_ecb.txt -K $KEY -nosalt  
2>/dev/null
```

```
openssl enc -aes-128-cbc -d -in plain_cbc_corrupt.bin -out dec_cbc.txt -K $KEY -iv $IV -nosalt  
2>/dev/null
```

```
openssl enc -aes-128-cfb -d -in plain_cfb_corrupt.bin -out dec_cfb.txt -K $KEY -iv $IV -nosalt  
2>/dev/null
```

```
openssl enc -aes-128-ofb -d -in plain_ofb_corrupt.bin -out dec_ofb.txt -K $KEY -iv $IV -nosalt  
2>/dev/null
```

compare

```
echo "ECB vs Original (first 60) "
```

```
head -c 60 plain.txt | hexdump -C
```

```
head -c 60 dec_ecb.txt | hexdump -C
```

```
echo " CBC vs Original "
```

```
head -c 60 dec_cbc.txt | hexdump -C
```

Task4-Padding

1.create file

```
printf "Hello, this is 17 bytes" > padding_test.txt
```

```
wc -c padding_test.txt → 23 bytes
```

2.Encryption

```
openssl enc -aes-128-cbc -e ... → cbc_padded.bin (32 bytes)
```

```
openssl enc -aes-128-ofb -e ... → ofb_nopad.bin (23 bytes)
```

Output:

23 padding_test.txt

32 cbc_padded.bin

23 ofb_nopad.bin

78 total

Task5:Message Digest

1.File create

```
echo " This is Task 5 of INS Lab. Learning how to generate message digests using OpenSSL." > digest_test.txt  
echo " This is Task 5 of INS Lab. Learning how to generate message digests using OpenSSL!" > digest_test_modified.txt
```

2.Hash Calculation

```
md5sum digest_test.txt
```

```
sha1sum digest _test.txt
```

```
sha256sum digest _test.txt
```

3.output

```
#MD5(digest_test.txt)= c3c2e8eea7fcc8ff564ac18a332ca037
```

```
8c207043e9817bcbdb1e8bf86e1eda90 digest_test_modified.txt
```

```
#SHA1(digest_test.txt)= 78a46f1fd4d8dfdfa7b95a653071cc157919e19a
```

```
e0b57f17bb64f1801245891caa72d4f6c984959d digest_test_modified.txt
```

```
#SHA2-256(digest_test.txt)=
```

```
72a0fd7ee7bb12557089ead0ca0de5508ac959725f0e3e62fac7b805b4dc0a6a
```

```
b5c9b4bfc33be09908487cd854db24d0ee5ea344abea3b1a935a5d420fea3759
```

```
digest_test_modified.txt
```

4. Observations

- Same data → Same hash
- Changing 1 bit → Completely different hash
- SHA-256 is the most secure

Task – 7: Keyed hash and HMAC

1. Files

-original.txt: "This is a test message for one-way hash property."

- modified.txt: First byte modified by flipping 1 bit ('T' → hex '54' → 'D4')

2.Hash Outputs

MD5

H1 (Original): f7a049b68141e64495d146134f5d8db0

H2 (Modified): 32c226d0792ac69a9cf8f377621aad5

SHA-256

H1 (Original): 4dbb6b5c595efc858bbd10e30b6f6a51371260a35c0d52328c5f7b2afd9847b1

H2 (Modified): 58338dbc2cc4d26bf25b265c100685c96ac274c74160ee706ec23913f6929360

3. Observations

=== MD5 Bit Difference ===

Same bits: 35/128 (27.34%)

=== SHA-256 Bit Difference ===

Same bits: 73/256 (28.52%)

5. Conclusion

"Hash functions exhibit the 'avalanche effect': changing 1 input bit causes ~20% of output bits to change. This proves the one-way and secure nature of hash functions."

